



Deep Learning

Mastering Hyperparameter tuning in Deep Learning

Dr. Mehran Safayani


safayani@iut.ac.ir


https://people.iut.ac.ir/en/safayani


https://www.aparat.com/mehran.safayani


https://github.com/safayani/deep_learning_course



Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

What Are We Tuning?

- Parameters (Learned internally):
- - Learned by the model during training from the data.
- - Examples: Weights (W), Biases (b).

- Hyperparameters (Set by You):
- - External configurations set before training begins.
- - Examples: Learning Rate, Batch Size, Number of Layers, Dropout Rate.

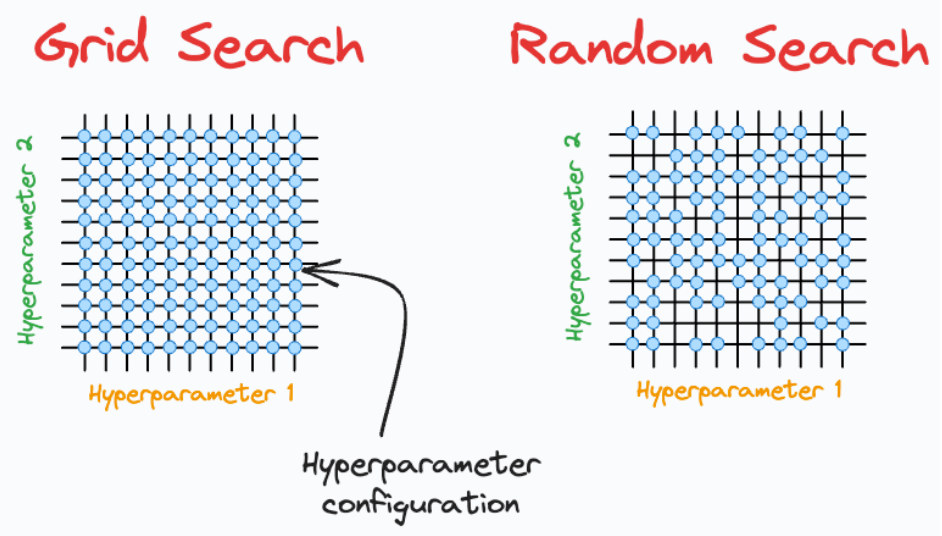




The Classics - Grid Search vs. Random Search

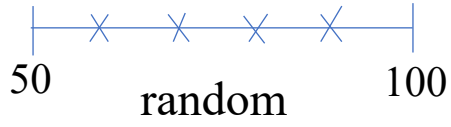
- Grid Search:
- - Exhaustive search through a manually specified subset.
- - Pros: Simple to implement, easily parallelizable.
- - Cons: 'Curse of Dimensionality' makes it computationally expensive.

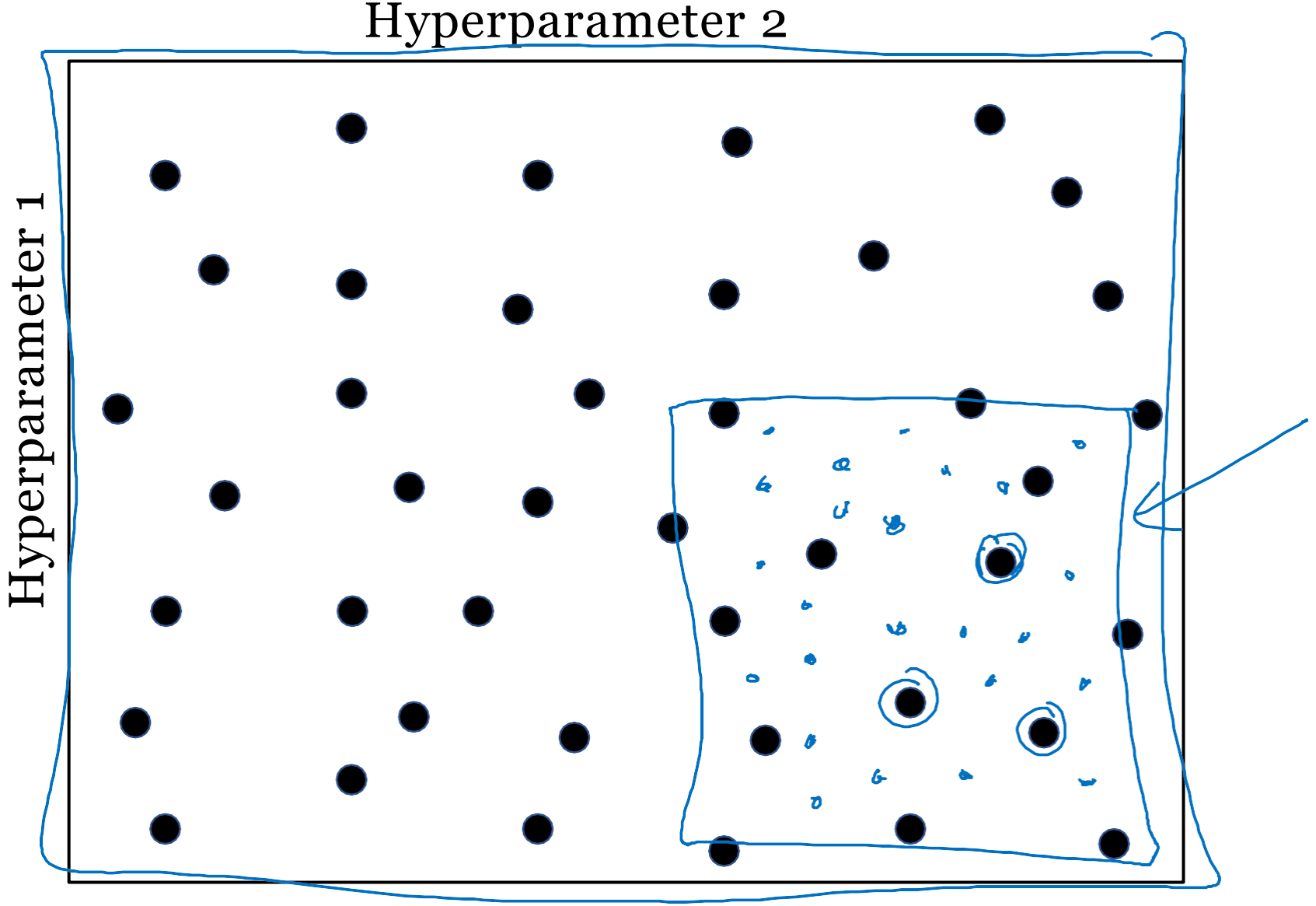
- Random Search:
- - Randomly sampling from a distribution over the hyperparameter space.
- - Pros: Proven to find better models in less time.
- - Cons: Uninformed; does not learn from previous iterations.



Coarse to fine

- $n^{[l]} = 50, \ldots, 100$





Using an appropriate scale to pick hyperparameters

$$n^{[l]} = 50, \ldots, 100$$

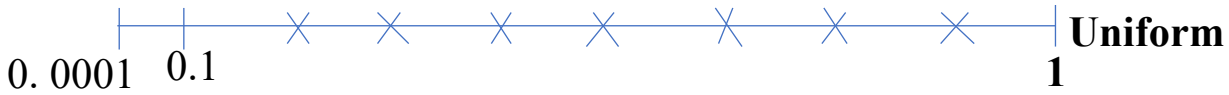


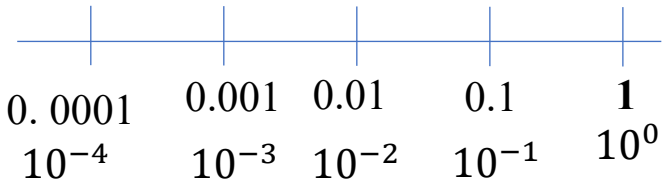
- #Layers

  L: 2 – 4

  2,3,4

- $\alpha = 0.0001, \ldots, 1$ $\quad \alpha = 10^r \; r = -1 * np \cdot random \cdot randint(4)$



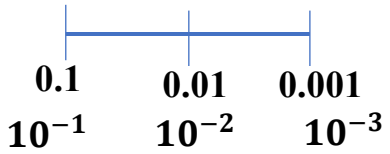






Using an appropriate scale to pick hyperparameters

- $r = -4 * np \cdot random \cdot rand()$ $\quad r \in [-4, 0]$
- $\alpha = 10^r$
- $\beta = 0.9 \cdots 0.999$
- $1 - \beta = 0.1 \cdots 0.001$



- $r \in [-3, -1]$

  $1 - \beta = 10^r$

  $\beta = 1 - 10^r$

Bayesian Optimization: Learning from the Past

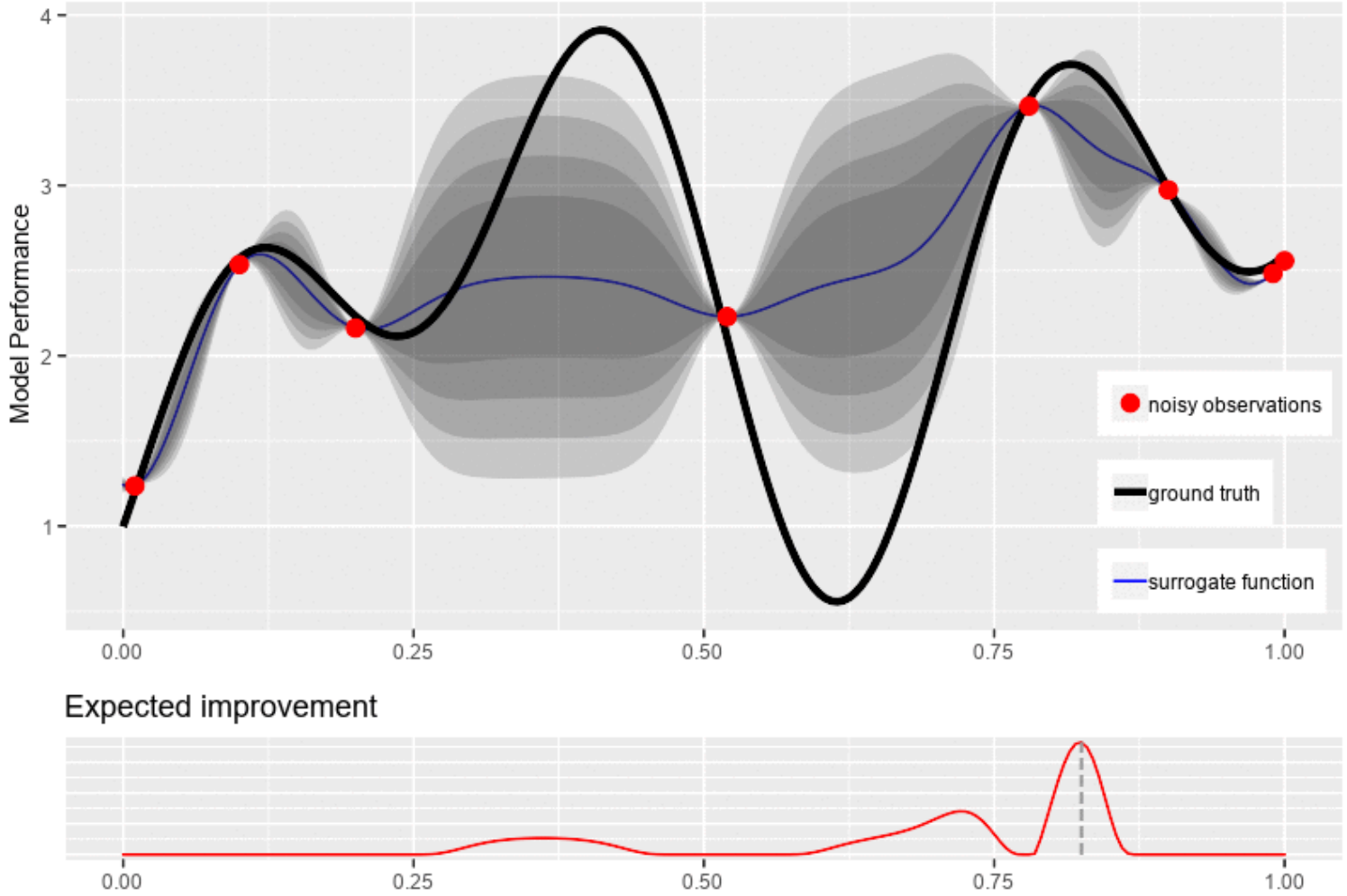
- Instead of guessing blindly, Bayesian Optimization builds a Surrogate Model.
- - Often modeled using a Gaussian Process (GP).
- - It tracks the mean (prediction) and variance (uncertainty).

- It uses an Acquisition Function to decide where to sample next by balancing:
- - Exploitation: Sampling where the model predicts a good result.
- - Exploration: Sampling where the model is highly uncertain.



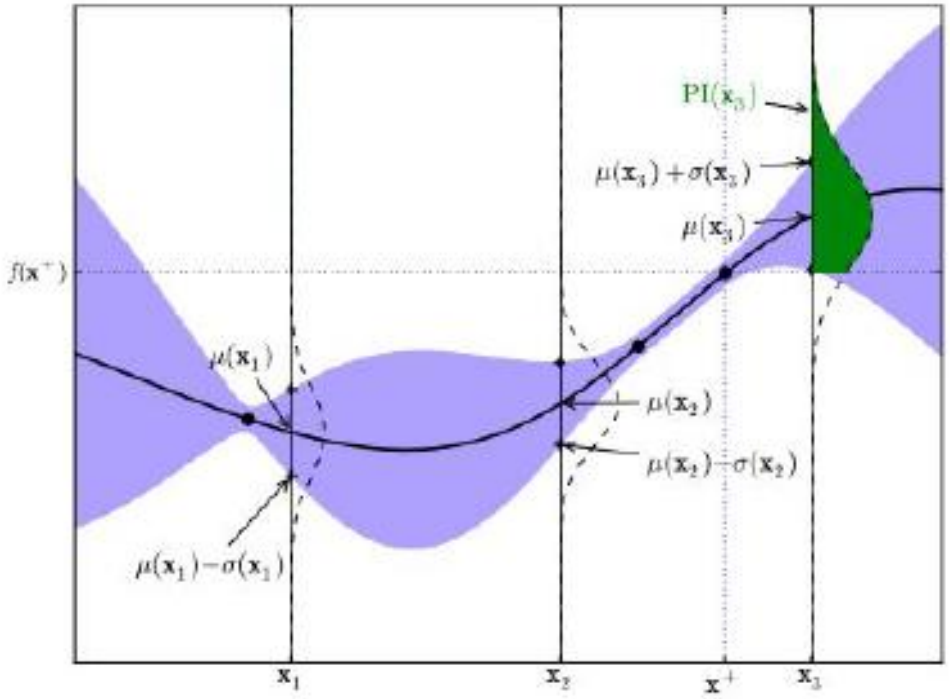


Bayesian optimization



The Acquisition Function: Expected Improvement (EI)

- Expected Improvement (EI) is the most common acquisition function.
- It calculates the expected amount of improvement over the current best observation.
- Formula: EI(x) = E[max(f(x) - f(x+), 0)]
- How it works in practice:
- - Peaks exactly where we should evaluate our next hyperparameter.
- - Prevents wasting compute on known bad regions.



The Heartbeat of Training: Learning Rate

- Determines the step size at each iteration while moving toward a minimum.

- Too High:
- - Divergence, skipping the minima entirely.

- Too Low:
- - Agonizingly slow convergence.
- - High risk of getting stuck in local minima or saddle points.





Evolution of Adaptive Learning Rates

- Adapting step size automatically per-parameter based on historical gradients: $G_{t,i} = \sum_{k=1}^{t} (g_{k,i})^2$
- - AdaGrad: Scales down LR for frequently updated parameters. $\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i}+\epsilon}} \cdot g_{t,i}$
- - RMSProp: Solves AdaGrad's vanishing LR using an exponential moving average.
- - Adam (Adaptive Moment Estimation): Combines RMSProp (variance) with Momentum (mean).

- Adam became the industry standard, but it had hidden flaws regarding weight decay.





State-of-the-Art: AdamW & Cosine Annealing

$$\text{Weight Update Magnitude} \propto \frac{\text{Original Gradient} + \lambda\theta}{\sqrt{v}}$$

- AdamW (Loshchilov & Hutter, 2019):

$$\theta_t = \theta_{t-1} - \eta \cdot \text{Adam_Calc}(\nabla\text{Loss} + \text{L2_Penalty})$$

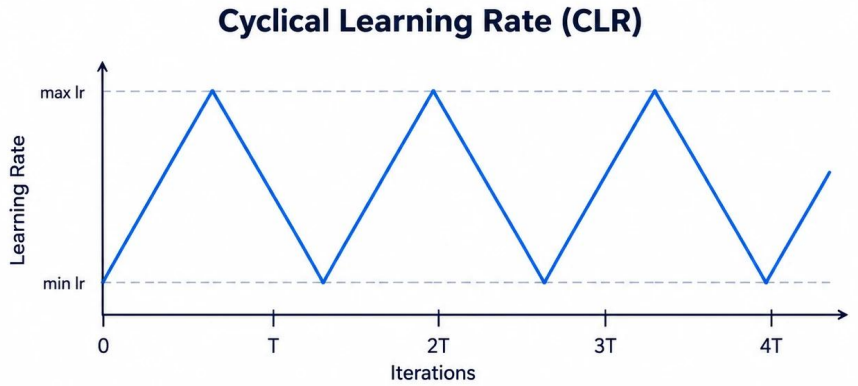
- Imagine you have two weights (w_1 and w_2):
- w_1 has **large, frequent gradients** (a very active feature).
- w_2 has **very small, rare gradients** (a rare feature).
- **The Problem with Adam + Coupled L_2:**
  - Adam divides updates by $\sqrt{v}$ (the running average of squared gradients) to stabilize learning:
  - Because w_1 has a large gradient, its denominator is large, which **shrinks its update**. As a result, the L_2 regularization penalty gets muted and fails to shrink w_1 properly.
  - For w_2 (small gradient), the denominator is tiny, which **amplifies its update**. The regularization penalty hits w_2 disproportionately hard, wrongly crushing it.

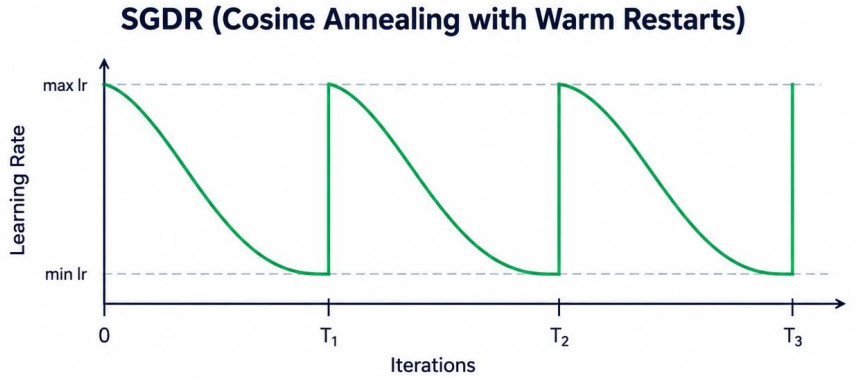
$$\theta_t = \theta_{t-1}(1 - \eta\lambda) - \eta \cdot \text{Adam_Calc}(\nabla\text{Loss})$$

- AdamW decouples weight decay, leading to much better generalization.

Advanced Learning Rate Schedules: CLR & SGDR

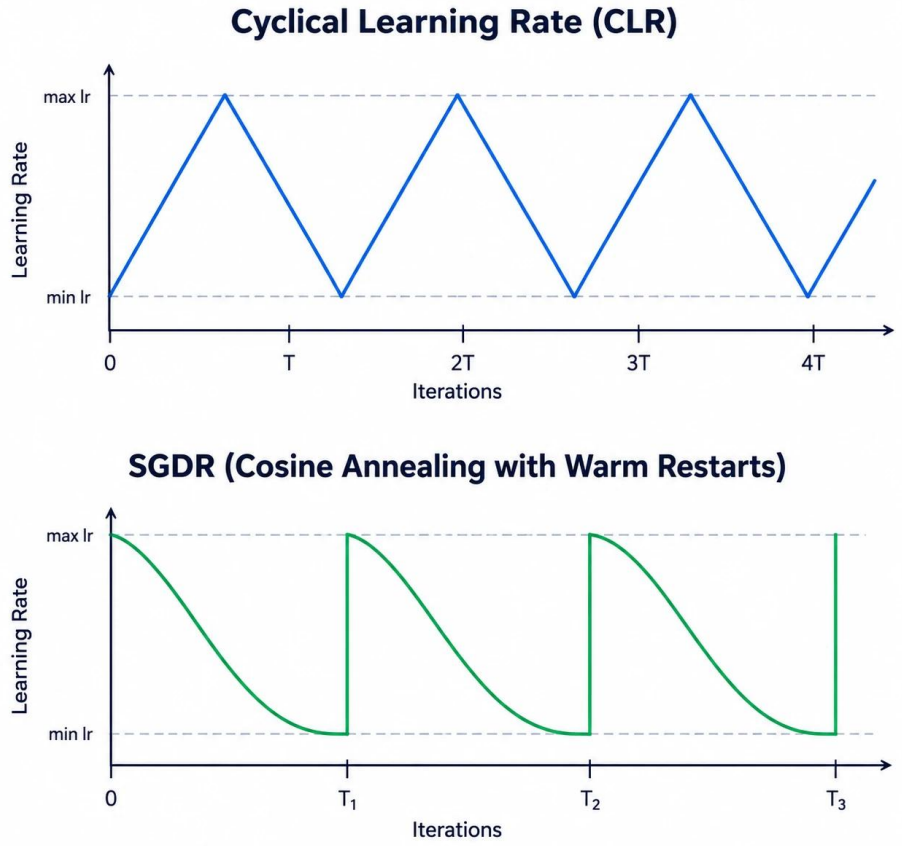
- **Beyond Monotonic Decay:**
  - Traditional schedules constantly decrease η.
  - Dynamic schedules use cyclical/periodic variations to explore the loss landscape more effectively.
- **1. Cyclical Learning Rates (CLR / Leslie Smith):**
  - **Mechanism:** Oscillates ηlinearly between a lower (η_{min}) and upper (η_{max}) bound (triangular/zigzag pattern).
  - **Why it works:**
    - **High LR phases:** Help the optimizer escape saddle points, flat regions, and sharp local minima.
    - **Low LR phases:** Enable fine-grained convergence in promising basins.
- **2. SGDR (Cosine Annealing with Warm Restarts):**
  - **Mechanism:** Decays ηfollowing a smooth cosine curve, then periodically resets ("warm restart") ηback to a higher value.
  - **Benefit:** Allows the model to jump out of suboptimal local minima and explore entirely new regions of the parameter space.
- **Key Takeaway:** Prevents premature convergence, improves generalization, and can drastically speed up training ("Super-convergence").





• In CLR & SGDR, How to find η_{min} and ∂_{max} (LR Range Test):

- Run a short training test, linearly increasing LR from a tiny value to a very large one over a few epochs.
- Plot Training Loss vs. Learning Rate.
- η_{max} **(Upper Bound):** The point just before the loss diverges or stops decreasing rapidly (often set to $\approx 1/3$ to $1/4$ of the divergence point).
- η_{min} **(Lower Bound):** Typically set to a fraction of η_{max} (e.g., $\eta_{max}/3$ or $\eta_{max}/10$).



The Frontier: LION (Evolved Sign Momentum)

- Discovered in 2023 by Google Brain using symbolic search algorithms.
- How it works:

$$\theta_t = \theta_{t-1} - \eta \cdot \text{sign}(c_t)$$

  - ct is an exponentially moving average of past gradients
  - Tracks momentum but uses only the SIGN (+1 or -1) of the gradient.
  - Drops the complex magnitude tracking of Adam.
- Why it matters:
- - Highly memory efficient (tracks fewer moments).
- - Outperforms AdamW in training Large Language Models (LLMs) and Vision Transformers.





Multi-Fidelity Tuning: Stop Wasting Compute

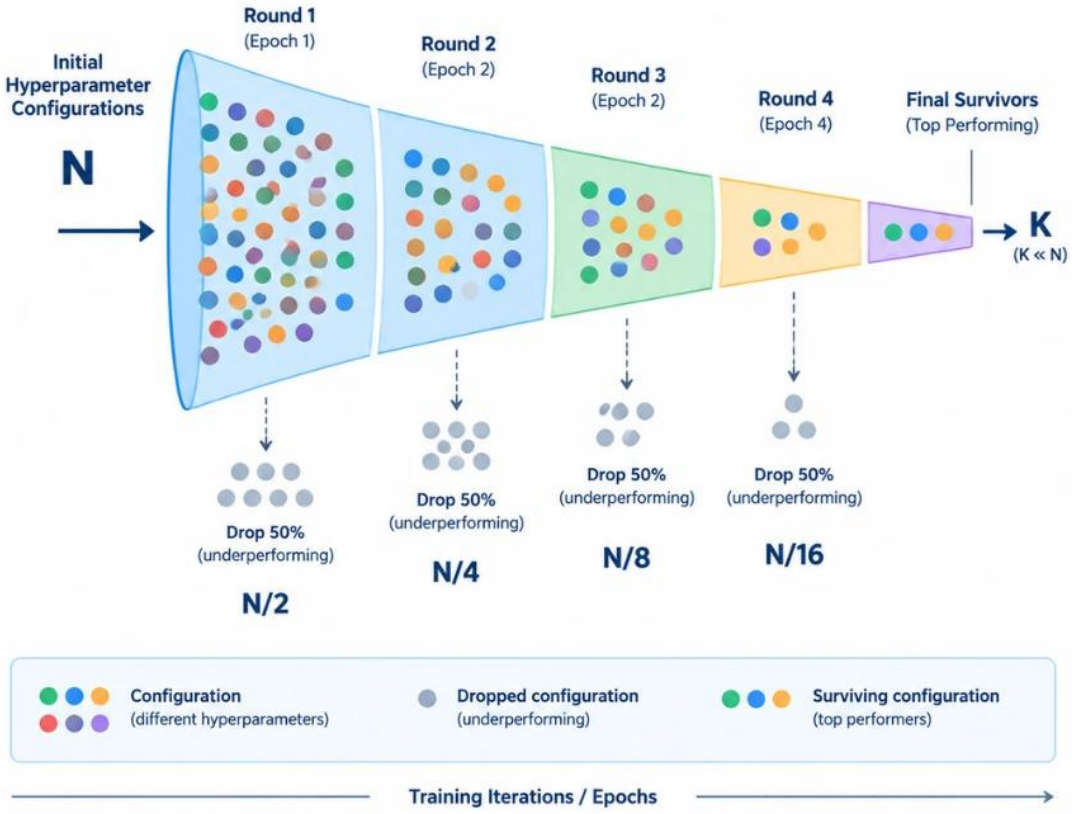
- Successive Halving
- Hyperband
- BOHB (Bayesian Optimization + Hyperband)





Successive Halving (SHA): The 'Fail Fast' Funnel

- Concept: Stop wasting compute on bad hyperparameters.
- 1. Start Broad: Test many configurations (e.g., 64) for a tiny budget (e.g., 1 epoch).
- 2. Evaluate & Prune: Rank models by validation loss. Discard the worst performing half.
- 3. Invest in Winners: Double the budget (e.g., 2 epochs) for the surviving models.
- 4. Repeat: Continue halving the pool and doubling the budget until 1 optimal model remains.
- The Flaw: Demands you perfectly balance 'Initial Budget' vs 'Number of Configurations' before starting. Slow-starting models are often killed prematurely.



Hyperband: Dynamic Resource Allocation

The Solution to SHA's Flaw: Instead of running one funnel, run multiple different funnels (Brackets).

Hyperband automatically divides your total compute budget into different brackets:

- Bracket 1 (Max Exploration): Starts with massive configs, tiny initial budget. High pruning.
  - *Goal:* Cast a massive net to find models that learn extremely fast.
- Bracket 2 (Balanced): Starts with moderate configs, moderate initial budget.
  - *Goal:* Give configurations a little more time to “warm up” before the first brutal cut
- Bracket 3 (Patient/Standard): Starts with few configs, full budget. Zero early pruning.
  - Goal: Protect “slow starter” models from being killed off too early

Why it wins: It mathematically guarantees you find the best tradeoff between exploring many configurations vs. giving complex models time to converge.





Hyperband Example: R=81, η =3

Bracket	Goal	Initial Configs	Successive Halving Path (Configs @ Epochs)
s=4	Max Exploration	81	81@1e → 27@3e → 9@9e → 3@27e → 1@81e
s=3	Balanced	34	34@3e → 12@9e → 4@27e → 1@81e
s=2	Intermediate	15	15@9e → 5@27e → 1@81e
s=1	Patient	8	8@27e → 2@81e
s=0	Pure Evaluation	5	5@81e (No Early Pruning)





BOHB: State-of-the-Art Tuning

Hyperband is great, but it samples configurations randomly.

Bayesian Optimization (BO) is smart, but it wastes time evaluating bad models to completion.

BOHB (Bayesian Optimization + Hyperband) marries the two:

1. Smarter Sampling (BO): BOHB builds a probabilistic model of the search space based on previous evaluations to pick highly promising configurations.

2. Cheaper Evaluation (Hyperband): It passes these smart configurations to Hyperband, which evaluates them quickly using multi-fidelity brackets.

Result: Strong anytime performance (Hyperband) with rapid convergence to the absolute global optimum (Bayesian Optimization).





Hyperband / BOHB Resource Allocation Table ($R = 81, \eta = 3$)

Bracket (s)	Round 0 (1 epoch)	Round 1 (3 epochs)	Round 2 (9 epochs)	Round 3 (27 epochs)	Round 4 (81 epochs)
Bracket 4 *(Max Exploration)*	81configs	27configs	9configs	3configs	1config
Bracket 3		34configs	11configs	3configs	1config
Bracket 2			15configs	5configs	1config
Bracket 1				8configs	2configs
Bracket 0 *(Max Exploitation)*					5configs





Tools of the Trade (2024 Landscape)

1. Dedicated Tuning Frameworks (The Engines)

- Optuna: The absolute industry standard. Define-by-run API, lightweight, supports BOHB/Hyperband.
- Ray Tune: Best for massive distributed cluster computing.
- KerasTuner: Best for native TensorFlow/Keras workflows.

2. Experiment Tracking & Visualization (The Dashboards)

- Comet (Comet.ml): Powerful hyperparameter sweeps, visual debugging, and seamless integration with Optuna/Ray.
- Weights & Biases (W&B): Extremely popular for Deep Learning sweeps and collaborative tracking.
- MLflow: The enterprise standard for full lifecycle management.

3. Cloud & AutoML Platforms (Fully Managed)

- Google Vertex AI: Serverless hyperparameter tuning (Vizier).
- AWS SageMaker: Automatic Model Tuning (AMT) for production pipelines.





Best Practices for the Real World

- 1. Start with AdamW or Lion, with a Cosine Annealing schedule.
- 2. Tune the Learning Rate and Batch Size first; they matter the most.
- 3. Avoid Grid Search. Use Random Search for small spaces, and Hyperband/Optuna for large spaces.
- 4. Always track your experiments systematically (e.g., MLflow, W&B).



